LLM Alignment

AI623 - Programming Assignment 2

This assignment is not graded and has no deliverables. There is no submission for this assignment, and no marks are awarded for producing code, plots, or results. Instead, the assessment of the PA content will occur through two separate components:

- **a viva** (oral examination), and
- **a written test** (a short quiz).

The purpose of this assignment is therefore *preparatory*: it is a structured way to build the understanding and implementation intuition that you will be asked to demonstrate in the viva and written test. **Please aim to complete this assignment by 2nd April 2026.**

Maintain a working document (strongly encouraged). Even though nothing is submitted, you are strongly encouraged to maintain a "working document" (notes + screenshots + plots + short explanations) as you progress.

Required reading before you start. You are strongly advised to go through the material in 2.3 before attempting the analytical or coding tasks. This includes a mix of articles on the intuition and theory of alignment, relevant sections from papers, and blog posts discussing pseudo-code to help you get started. You are also provided with some repositories in 1 that can serve as guides for your code. The intent is not to copy those implementations from these frameworks, but to understand them and apply the optimizations and structure within to your own code.

Contents

1 Allowed Tools and Constraints **3**

2 Background and Notation **4**
2.1 Notation Dictionary . . . 4
2.2 The Alignment Pipeline . . . 4
2.3 Resources . . . 5
2.3.1 Alignment Literature . . . 5
2.3.2 Background Reading . . . 5
2.3.3 LoRA and PEFT . . . 5

3 Analytical Section **6**
3.1 Problem 1: The RLHF Objective and the Optimal Policy . . . 6
3.2 Problem 2: Proximal Policy Optimization (PPO) for LLMs . . . 6
3.3 Problem 3: Direct Preference Optimization (DPO) . . . 8
3.4 Problem 4: Group Relative Policy Optimization (GRPO) . . . 8
3.5 Problem 5: Synthesis and Comparison . . . 9

4 Coding Section **11**
4.1 Dataset and Task . . . 11
4.2 Baseline Configuration . . . 12
4.3 Recommended Code Organization . . . 12
4.4 Task C0: Data Pipeline and Model Loading . . . 13
4.5 Task C1: Reward Model Training . . . 14
4.6 Task C2: Supervised Fine-Tuning (SFT) Warm-Up . . . 15
4.7 Task C3: PPO Implementation . . . 16
4.8 Task C4: DPO Implementation . . . 18
4.9 Task C5: GRPO Implementation . . . 19
4.10 Task C6: RLVR — RL with Verifiable Rewards . . . 20
4.11 Task C7: Focused Ablations . . . 23
4.12 Task C8: Evaluation . . . 23

5 Appendix **25**
5.1 Mathematical Reference . . . 25
5.2 VRAM Budget (Approximate, bfloat16) . . . 25
5.3 Debugging Checklist . . . 26

1 Allowed Tools and Constraints

- You may use **PyTorch**, **HuggingFace Transformers** (for loading models, tokenizers, `AutoModel*` classes, and datasets), **PEFT** (for LoRA adapters), **Accelerate** (for device placement), and standard scientific Python libraries.
- You may **not** use high-level RL or alignment libraries: **no TRL, trlX, OpenRLHF, RL4LMs, or any library that provides a ready-made PPO/DPO/GRPO trainer**. The alignment logic must be yours—reward computation, advantage estimation, policy gradients, clipping, preference losses. However, you can and are expected to use these repositories as references before you start coding.
- You **may** load and freeze a pre-trained model from the SmolLM family (one of SmolLM2-360M, SmolLM2-1.7B, SmolLM3-3B-Base, whatever maximum your compute and memory budget allows) as the initial policy and reference model. You must implement all RL/DPO update logic from scratch.
- For the reward model, you **must** use `AutoModelForSequenceClassification` (from HuggingFace Transformers) initialized from the same base checkpoint and fine-tune it on the preference dataset. You must implement the training loop yourself (no Trainer API).
- `bitsandbytes` quantization (`load_in_8bit` or `load_in_4bit`) is **allowed** to fit models on GPU.

2 Background and Notation

2.1 Notation Dictionary

Symbol	Meaning
x	Prompt / input
$y = (y_1, \ldots, y_T)$	Completion / response (token sequence)
$\pi_\theta(\cdot \mid s)$	Policy: distribution over next token given state s
π_{ref}	Reference (SFT) policy, frozen
π_θ^{old}	Old policy snapshot used to collect roll-outs
$r_\psi(x, y)$	Scalar reward from reward model r_ψ
$V_\phi(s_t)$	Value function / critic
$s_t = (x, y_{<t})$	State at step t: prompt + tokens generated so far
$a_t = y_t$	Action at step t: next token sampled
R_t	Discounted return from step t onwards
A_t	Advantage at step t
δ_t	TD error at step t
$\rho_t(\theta)$	Probability ratio $\pi_\theta(y_t \mid s_t)/\pi_\theta^{\text{old}}(y_t \mid s_t)$
D_p	Prompt pool
D_{pr}	Preference dataset of triples (x, y^+, y^-)
β	KL penalty coefficient
ε	PPO clipping parameter
γ, λ	Discount and GAE decay factors

2.2 The Alignment Pipeline

A language model is *aligned* via the following three-stage pipeline:

1. **Pre-training**: Θ_{PLM} learned on large corpora. (Provided: selected model from SmolLM family.)

2. **Supervised Fine-Tuning (SFT)**: Θ_{SFT} obtained by fine-tuning on curated instruction-following data. (You will do a lightweight LoRA SFT in Task C1.)

3. **Preference Optimization (PO)**: Θ_{aligned} obtained by further optimizing π_θ (initialized from Θ_{SFT}) using PPO, DPO, GRPO, or RLVR.

2.3 Resources

2.3.1 Alignment Literature

1. **RLHF (InstructGPT)** — Ouyang et al., "Training language models to follow instructions with human feedback", NeurIPS 2022. (Section 3)
2. **PPO** — Schulman et al., "Proximal Policy Optimization Algorithms", arXiv 2017. (Section 3 (the clipped surrogate) and Algorithm 1)
3. **GAE** — Schulman et al., "High-Dimensional Continuous Control Using Generalized Advantage Estimation", ICLR 2016. (Sections 3–4)
4. **DPO** — Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model", NeurIPS 2023. (Sections 3–5)
5. **GRPO** — Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models", arXiv 2024. (Section 4)
6. **RLVR / DeepSeek-R1** — DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning", arXiv 2025. (Section 2–3 and Appendix A)

2.3.2 Background Reading

1. Hugging Face, "Illustrating Reinforcement Learning from Human Feedback (RLHF)".
   This is a high-level walkthrough of the full alignment pipeline, which you should read before diving into the papers.
2. **OpenAI spinning up in deep RL**
   This is a good reference that includes pseudocode and worked-out explanations of PPO's update rule.
3. **"RLHFlow" (repo and blogs)**
   - Bradley-Terry Model
   - Alignment Handbook

2.3.3 LoRA and PEFT

1. **Understanding LoRA (Low-Rank Adaptation)** – Vikram Pande.
2. **LoRA: Low-Rank Adaptation of Large Language Models** – Hu et al.
3. **Fine-Tuning vs PEFT: Parameter Efficient Fine-Tuning – A Practical Guide** – Amit.
4. **Motivation for Parameter-Efficient Fine-Tuning for LLMs** – M. Brenndoerfer.
5. **LoRA / PEFT Explanation (YouTube Video)**
6. **Disabling and re-enabling adapters**
7. **Saving and loading LoRA adapters**
8. **Merging adapters for inference**

3 Analytical Section

3.1 Problem 1: The RLHF Objective and the Optimal Policy

We work in the standard RLHF setting. Let $x \sim D_p$ be a prompt and $y \sim \pi_\theta(\cdot \mid x)$ be a completion. A frozen reward model $r_\psi(x, y)$ scores completions. The base RLHF objective is:

$$\max_\theta \; J_B(\theta) = \mathbb{E}_{x \sim D_p,\, y \sim \pi_\theta}\big[r_\psi(x, y)\big]. \tag{1}$$

To prevent reward hacking and preserve knowledge in $\pi_{\text{ref}} = \pi_{\Theta_{\text{SFT}}}$, a KL penalty is added:

$$\max_\theta \; J(\theta) = \mathbb{E}_{x \sim D_p,\, y \sim \pi_\theta}\big[r_\psi(x, y)\big] - \beta\, \mathbb{E}_{x \sim D_p}\big[\mathrm{KL}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\big)\big]. \tag{2}$$

Problem 1.1 (Optimal policy in closed form)

Show that the optimal solution to (2) is:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\, \pi_{\text{ref}}(y \mid x)\, \exp\Big(\tfrac{1}{\beta} r_\psi(x, y)\Big), \quad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\, e^{r_\psi(x,y)/\beta}. \tag{3}$$

Hint: Write the KL term explicitly, treat as a constrained variational problem, and solve pointwise over y.

Problem 1.2 (Reward as log-ratio)

Rearrange (3) to show:

$$r_\psi(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{4}$$

Explain in words why this identity is the key insight behind DPO (Problem 3).

Problem 1.3 (Bradley-Terry preference model)

Human preference data is modeled with the Bradley-Terry model. Given a pair (y^+, y^-) where y^+ is preferred over y^- for prompt x:

$$P(y^+ \succ y^- \mid x) = \sigma\big(r_\psi(x, y^+) - r_\psi(x, y^-)\big), \tag{5}$$

trained via:

$$\mathcal{L}_{\text{RM}}(\psi) = -\mathbb{E}_{(x, y^+, y^-) \sim D_{\text{pr}}}\big[\log \sigma\big(r_\psi(x, y^+) - r_\psi(x, y^-)\big)\big]. \tag{6}$$

(a) **Analyse.** The reward model is trained on (x, y^+, y^-) pairs collected from a fixed behavior policy π_b that only covers a small region of response space. What pathologies can arise when this reward model is used to optimize π_θ via (2)? Connect them to specific terms in (2).

3.2 Problem 2: Proximal Policy Optimization (PPO) for LLMs

In PPO, the LM policy π_θ is an agent selecting tokens sequentially. The trajectory τ corresponds to generating a full completion y given prompt x.

Per-Token Rewards

The sparse sequence-level reward $r^{\text{task}} = r_\psi(x, y)$ is received only at $t = T$. A KL-shaping term is added at each step:

$$r_{i,t}^{\text{KL}} = -\beta\Big[\log \pi_\theta^{\text{old}}(a_t \mid s_t) - \log \pi_{\text{ref}}(a_t \mid s_t)\Big]. \tag{7}$$

Total per-step reward: $r_{i,t} = r_{i,t}^{\text{task}} + r_{i,t}^{\text{KL}}$.

Advantage Estimation via GAE

$$\delta_t = r_t + \gamma V_\phi^{\text{old}}(s_{t+1}) - V_\phi^{\text{old}}(s_t), \qquad A_t^{\text{GAE}(\lambda)} = \sum_{k=0}^{T-t} (\gamma\lambda)^k \delta_{t+k}. \tag{8}$$

PPO Clipped Objective

Let $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_\theta^{\text{old}}(a_t \mid s_t)$.

$$L_{\text{clip}}(\theta) = \mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\,\tilde{A}_t,\ \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\tilde{A}_t\Big)\Big]. \tag{9}$$

Critic loss: $L_V(\phi) = \frac{1}{U}\sum_{i,t}\big(V_\phi(s_{i,t}) - V_\phi^{\text{GAE}}(s_{i,t})\big)^2$. Full PPO loss: $L_{\text{PPO}}(\theta, \phi) = -L_{\text{clip}}(\theta) + L_V(\phi) + L_{\text{KL}}(\theta) - L_{\text{ENT}}(\theta)$.

Problem 2.2 (PPO clipping mechanics)

For each of the six regions below, determine the return value of $\min(\cdot)$, whether it is clipped, and whether $\nabla_\theta L_{\text{clip}} = 0$.

Region	ρ_t range	$\tilde{A}_t$	$\min(\cdot)$ value	Clipped?	$\nabla = 0$?
(1)	$[1-\varepsilon,\ 1+\varepsilon]$	> 0			
(2)	$[1-\varepsilon,\ 1+\varepsilon]$	< 0			
(3)	$< 1-\varepsilon$	> 0			
(4)	$< 1-\varepsilon$	< 0			
(5)	$> 1+\varepsilon$	> 0			
(6)	$> 1+\varepsilon$	< 0			

Identify which regions block the gradient and explain why this is desirable.

Problem 2.3 (Reward hacking, credit assignment, and the critic)

(a) Explain the two core issues with maximizing (1) without KL regularization: (i) reward hacking from the distribution shift, (ii) credit assignment with sparse rewards. Connect each to a specific term in the objective.

(b) Why is the per-token KL term $r_{i,t}^{\text{KL}}$ more informative than a single sequence-level KL penalty?

3.3 Problem 3: Direct Preference Optimization (DPO)

DPO bypasses explicit reward modeling by reparameterizing the reward via (4). The DPO loss for a single pair (x, y^+, y^-) is:

$$\begin{aligned}\ell_{\text{DPO}}(\theta) &= -\log\sigma\left(\beta\log\frac{\pi_\theta(y^+\mid x)}{\pi_{\text{ref}}(y^+\mid x)} - \beta\log\frac{\pi_\theta(y^-\mid x)}{\pi_{\text{ref}}(y^-\mid x)}\right)\\ &= -\log\sigma(z), \quad z = \beta\big(\Delta_\theta - \Delta_{\text{ref}}\big),\end{aligned} \tag{10}$$

where $\Delta_\theta = \log\pi_\theta(y^+\mid x) - \log\pi_\theta(y^-\mid x)$ and $\Delta_{\text{ref}} = \log\pi_{\text{ref}}(y^+\mid x) - \log\pi_{\text{ref}}(y^-\mid x)$.

Problem 3.1 (Deriving the DPO loss)

(a) Starting from the Bradley-Terry model (5) and the reward reparameterization (4), derive the DPO loss (10) step by step. Be explicit about where $\log Z(x)$ cancels.

(b) Since $\log\pi_\theta(y\mid x) = \sum_t \log\pi_\theta(y_t\mid x, y_{<t})$, explain why longer sequences receive larger gradient signals, and why this creates a *verbosity bias*.

(c) Show that the DPO loss has the same structure as $\mathcal{L}_{\text{RM}}$ in (6). What does this reveal about the implicit reward learned by DPO?

Problem 3.2 (DPO gradient analysis)

Show that:

$$\frac{\partial\ell_{\text{DPO}}}{\partial\log\pi_\theta(y^+\mid x)} = -\beta(\sigma(z)-1), \qquad \frac{\partial\ell_{\text{DPO}}}{\partial\log\pi_\theta(y^-\mid x)} = +\beta(\sigma(z)-1). \tag{11}$$

(a) When does the gradient nearly vanish? Interpret as a regularization effect.

(b) **Analyse.** Suppose π_{ref} assigns very low probability to both y^+ and y^- (e.g., it was fine-tuned on unrelated data). How does this corrupt Δ_{ref} and cause pathological training dynamics?

3.4 Problem 4: Group Relative Policy Optimization (GRPO)

GRPO replaces the critic with a group-computed baseline. For prompt x_b, K completions $\{y_{b,k}\}_{k=1}^K$ are sampled. Each is scored: $r_{b,k} = r_\psi(x_b, y_{b,k})$. Group mean $\mu_b = \frac{1}{K}\sum_k r_{b,k}$; group-relative advantage $A_{b,k} = r_{b,k} - \mu_b$.

$$L_{\text{GRPO}}(\theta) = -\frac{1}{K}\sum_{k=1}^{K}\frac{1}{T_k}\sum_{t=1}^{T_k}\min\Big(\rho_{k,t}A_{k,t},\ \text{clip}(\rho_{k,t}, 1-\varepsilon, 1+\varepsilon)A_{k,t}\Big) + \beta\text{KL}(\pi_\theta\|\pi_{\text{ref}}). \tag{12}$$

Problem 4.1 (GRPO vs. PPO: what is eliminated)

(a) GRPO eliminates V_ϕ. State two concrete computational/memory advantages for large-scale LLM fine-tuning.

(b) GRPO broadcasts $A_{b,k}$ to all tokens in completion k. What critical assumption does this make, and why is it weaker than token-level advantages?

(c) **Analyse.** When all K completions receive the same reward, $A_{b,k} = 0$ and the gradient vanishes. Describe the practical impact on (i) trivial prompts and (ii) very hard prompts. What does this imply about prompt-difficulty distribution in the training set?

Problem 4.2 (Length-invariance)

(a) Show that a short correct answer (T_k small, $A_{b,k} > 0$) gets a stronger per-token gradient push than an equally correct long answer. Similarly show a long incorrect answer is under-penalized.

(b) Explain why this causes GRPO to generate increasingly long sequences for incorrect outputs (optimization bias).

Problem 4.3 (RLVR: RL with Verifiable Rewards)

In RLVR, the learned r_ψ is replaced by a verifiable reward $r_v \in \{0, 1\}$ (checking arithmetic correctness, passing unit tests, etc.). DeepSeek-R1, Kimi K1.5, and Qwen3 all use RLVR.

(a) State two properties of verifiable rewards that make them preferable to learned reward models for reasoning tasks.

(b) For long completions, most tokens get gradient zero (before the final answer is verified). What does this imply for sample efficiency of RLVR with long reasoning traces?

(c) **Analyse.** Suppose the checker r_v is occasionally wrong (false positive/negative). How does reward noise propagate into the group-relative advantage? Under what conditions is GRPO more robust to verifier noise than PPO with a learned V_ϕ?

3.5 Problem 5: Synthesis and Comparison

Problem 5.1 (PPO vs. DPO vs. GRPO: a unified view)

Fill in the comparison table. Give a precise answer (one sentence or formula) per cell.

	PPO	DPO	GRPO
Requires explicit reward model?			
Requires critic/value model?			
Online vs. offline			
Advantage/signal source			
Main regularization			
Key failure mode			

Problem 5.2 (Alignment/RLHF failure modes)

For each failure mode, identify which method(s) it affects and propose a mitigation.

(a) **Reward hacking:** Policy exploits quirks in r_ψ due to distribution shift.

(b) **Mode collapse:** Probability mass concentrates on narrow high-reward patterns.

(c) **KL spikes:** Policy moves far from Θ_{SFT}, losing general capabilities.

(d) **Length/verbosity bias:** Trained models produce excessively long/repetitive outputs.

(e) **OOD generalization:** Alignment training uses "nice" prompts; deployment involves multi-turn, tool use, and novel topics.

4 Coding Section

4.1 Dataset and Task

You will use the **Anthropic HH-RLHF** dataset (`Anthropic/hh-rlhf` on HuggingFace Datasets). This dataset contains human preference triples (x, y^+, y^-) for helpfulness and harmlessness conversations.

Split	**Use**
`train` (harmless, ∼42k)	RM fine-tuning, SFT, PPO/GRPO prompt pool, DPO training
`test` (harmless, ∼2.3k)	Held-out evaluation for all methods

Preprocessing: Each example has a `chosen` and `rejected` field containing the full conversation (prompt + response concatenated). You must parse these into (x, y^+, y^-) triples.

4.2 Baseline Configuration

Baseline Configuration

You should run at least one experiment for each task. Below is a recommended configuration. You can adjust it according to your compute and memory budget (and whatever works best for you).

- **Policy model:** your chosen SmolLM2/SmolLM3 variant.
- **Reward model and value backbone:** `meta-llama/Llama-3.2-1B-Instruct` (`bfloat16`), loaded separately for the RM (Task C1) and value head (Task C3).
- **LoRA config (policy SFT/PPO/GRPO/DPO/RLVR):** rank $r = 8$, $\alpha = 16$, target modules `q_proj, v_proj`, dropout = 0.05.
- **Tokenizer:** model's own tokenizer; pad token = eos token; padding side = left.
- **Max sequence length:** 1024 tokens (prompt + response).
- **SFT:** 1 epoch over `chosen` responses; cross-entropy on response tokens only; batch 8 + grad accum 4 (effective 32).
- **RM fine-tuning:** 1 epoch; batch 8; margin ranking loss.
- **PPO/GRPO:** 200 update steps; 8 prompts per step; $K = 4$ roll-outs (GRPO); max new tokens 128; $\beta = 0.1$; $\varepsilon = 0.2$.
- **DPO:** 1 epoch; batch 8 + grad accum 4; $\beta = 0.1$.
- **RLVR:** 300 update steps on GSM8K; 8 prompts per step; $K = 4$ roll-outs; max new tokens 256; $\beta = 0.05$; $\varepsilon = 0.2$.
- **Evaluation:** Every 25 steps: RM score on 200 held-out prompts (greedy), KL from π_{ref}, win-rate vs. SFT baseline (RM judge).

4.3 Recommended Code Organization

Implementation Guidance

- `data/` — dataset loading, parsing HH-RLHF, preference pair construction, GSM8K loading and answer extraction.
- `model/` — model loading utilities (SmolLM2/SmolLM3 policy, Llama-3.2-1B reward/value), LoRA setup, value head, reward head.
- `alignment/ppo.py` — rollout, GAE, clipping, critic loss, full PPO update.
- `alignment/dpo.py` — log-prob computation, DPO loss.
- `alignment/grpo.py` — group rollout, group-relative advantage, GRPO update.

- `alignment/rlvr.py` — verifiable reward checker, RLVR training loop (GRPO variant with r_v).
- `train_sft.py` — SFT warm-up loop.
- `train_rm.py` — reward model fine-tuning loop.
- `train_rl.py` — PPO/GRPO/DPO/RLVR training loop + evaluation.
- `eval.py` — win-rate, KL computation, sample generation table, GSM8K pass@1.

4.4 Task C0: Data Pipeline and Model Loading

Task C0: Setup

Objective: Set up data loading, model loading, and LoRA configuration. All subsequent tasks depend on this.

C0.1 Data Loading and Parsing

1. Load `Anthropic/hh-rlhf` (harmless split) using `datasets.load_dataset`.
2. Implement a parser to parse the dataset's examples → `(prompt, chosen, rejected)` triplets. The `chosen` and `rejected` fields contain full conversation strings.
3. Build three dataloaders: SFT (prompt + chosen response), RM (preference pairs), DPO (preference pairs with both responses tokenized).
4. **Verify:** print 3 parsed triples and confirm the prompt/response split is correct.

C0.2 Model and Tokenizer Loading

5. Load your chosen SmolLM2/SmolLM3 policy model in `bfloat16`:
6. Set `tokenizer.pad_token = tokenizer.eos_token` and `tokenizer.padding_side = ''left''`.
7. Also load the **Llama-3.2-1B-Instruct** base model separately for use as the reward model and value model backbone (Task C1 and Task C3): Note: Llama-3.2 family requires accepting the model license on HuggingFace and logging in via `huggingface-cli login` (or setting `HF_TOKEN` in your Kaggle/Colab secrets).
8. Enable gradient checkpointing on the policy.
9. **Verify:** print parameter counts and GPU memory footprint for both the policy and the Llama-3.2-1B backbone using `torch.cuda.memory_allocated()`. Confirm both fit in your available VRAM before proceeding.

C0.3 LoRA Setup

10. Using `peft.LoraConfig` and `peft.get_peft_model`, apply LoRA to select layers (it is up to you to decide on where and to which layers you add LoRA) with $r = 8$, $\alpha = 16$.

11. Implement a function that creates a *frozen copy* of the model without LoRA (or with LoRA adapters disabled via `disable_adapters`).

Common Pitfalls & Debugging

- **Padding side matters**: for decoder-only generation, left-padding ensures all sequences in a batch end at the same position. Getting this wrong will corrupt generation and log-prob computation.
- **Gradient checkpointing + LoRA**: call `model.enable_input_require_grads()` after enabling gradient checkpointing when using PEFT, otherwise gradients may not flow through the LoRA layers.
- **Reference model**: π_{ref} must share no gradient state with π_θ. Use `model.disable_adapter_layers()` or load a separate copy and call `.requires_grad_(False)` on all parameters.

4.5 Task C1: Reward Model Training

Task C1: Train the Reward Model

Objective: Fine-tune a reward model $r_\psi(x, y) \in \mathbb{R}$ using `meta-llama/Llama-3.2-1B-Instruct` as the backbone and `AutoModelForSequenceClassification` to attach a scalar regression head. This model is kept *entirely separate* from the policy.

C1.1 Model Architecture

1. Load the reward model backbone as a sequence classifier using `AutoModelForSequenceClassification`. This replaces the LM head with a linear projection $\mathbb{R}^{d_{\text{model}}} \to \mathbb{R}^1$ (one scalar per input).
2. Apply LoRA to the reward model backbone (same rank/alpha as policy LoRA) to reduce memory during training. Since this model is frozen after training, LoRA mainly helps during the fine-tuning step. You can also simply just train the linear projection layer.

C1.2 Reward Model Loss

3. For a batch of (x, y^+, y^-) triples, tokenize `prompt + chosen` and `prompt + rejected` separately using `rm_tokenizer` (the Llama-3.2-1B tokenizer, which may differ from the policy tokenizer).
4. Forward pass: obtain $r^+ = r_\psi(x \oplus y^+)$ and $r^- = r_\psi(x \oplus y^-)$ from the scalar output at the last non-pad token position.
5. Implement the **margin ranking loss**:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}\big[\log \sigma(r^+ - r^-)\big] + \lambda_{\text{reg}} \cdot \mathbb{E}\big[(r^+)^2 + (r^-)^2\big],$$

where $\lambda_{\text{reg}} = 10^{-3}$ prevents reward magnitude growth.

6. Train for 1 epoch. Log: loss, preference accuracy ($r^+ > r^-$).

C1.3 Reward Model Evaluation

7. Evaluate on the held-out test set. Report:
   - Preference accuracy: $\mathbb{P}(r^+ > r^-)$ on test pairs.
   - Reward distribution: histograms of r^+ and r^- on test data.
8. **Target:** $\geq 60\%$ preference accuracy before proceeding.
9. Save the trained reward model as a **frozen artifact**. It is not updated during any subsequent RL training.

Implementation Guidance

Which token to read the scalar from for Llama-3.2-1B-Instruct: Llama is a decoder-only model. `AutoModelForSequenceClassification` by default reads the hidden state at the last token (index -1 in the sequence dimension) and projects it to a scalar. For left-padded sequences, this is always the final *real* token—but since we use right-padding for the RM (`padding_side=''right''`), We need to find the last non-pad token manually. Verify that `rm.config.pad_token_id` is set correctly.
Policy and Reward Model Tokenizers The policy uses the SmolLM2/SmolLM3 tokenizer; the reward model uses the Llama-3.2-1B tokenizer. This is fine since the RM scores the raw text, so tokenization differences between policy and RM are expected and handled by re-tokenizing the generated string before passing to the RM.

Common Pitfalls & Debugging

- Concatenate prompt *and* response before tokenizing for the RM: it scores the full (x, y) pair, not y in isolation.
- Do not use the `Trainer` API. Write the training loop explicitly.

4.6 Task C2: Supervised Fine-Tuning (SFT) Warm-Up

Task C2: SFT Warm-Up

Objective: Fine-tune the SmolLM2/SmolLM3 policy on HH-RLHF chosen responses or some other instruction following fine-tuning dataset like Databricks Dolly-15k or Alpaca before RL. This produces π_{ref} (frozen KL anchor) and the initial π_θ (trainable).

1. Implement the SFT Loss Function using the `CausalLM` forward pass. **Mask the prompt tokens** in the loss: only compute cross-entropy on the response tokens.
   - Set `labels[prompt_mask] = -100` (ignored by cross-entropy).
2. Train for 1 epoch with the baseline config (batch 8, grad accum 4).

3. Log: training loss per step, perplexity on held-out chosen responses every 100 steps.
4. After training, generate 5 sample responses (greedy) and verify they are coherent and on-topic. If responses are degenerate (repeated tokens, empty), debug before proceeding.
5. Save this checkpoint as **both**:
   - π_{ref} (frozen, LoRA adapters disabled) — used as KL anchor.
   - $\pi_\theta^{(0)}$ (trainable) — starting point for PPO/GRPO/DPO.

NOTE: You are allowed to adjust the subset you use for training according to your compute and memory budget.

Common Pitfalls & Debugging

A common bug: applying loss on the prompt tokens. If the model's perplexity is unusually low (e.g. < 5) from step 1, you are likely computing loss over both prompt and response. Check your label masking.

4.7 Task C3: PPO Implementation

Task C3: Proximal Policy Optimization

Objective: Implement RLHF with PPO for the 1B LLM.

C3.1 Value Head

1. The value model uses the **Llama-3.2-1B-Instruct** backbone (the same base checkpoint used for the reward model) with a scalar head added on top. This keeps the value model architecturally separate from the policy, avoiding gradient interference and matching the realistic setup where the critic is a distinct model.
2. The value head is a single linear layer $\mathbb{R}^{d_{\text{model}}} \to \mathbb{R}$. Initialize it with near-zero weights to avoid corrupting early training: `nn.init.normal_(value_head.weight, std=0.01)`.
3. The value backbone can be LoRA-adapted. Alternatively, freeze it and only train the scalar head— you can try both and report which is more stable and gives better results.
4. The value model is updated jointly during the PPO loss step. Its parameters are optimized separately from the policy's LoRA adapters.

C3.2 Roll-out Collection

5. Implement a function for rollout that does the following:
   - Generate a response y using `model.generate()` with sampling (temperature $= 0.7$, top-$p = 0.9$).

- Records `response_ids`, per-token log-probs under π_θ^{old}, per-token log-probs under π_{ref}, per-token value estimates $V_\phi(s_t)$.
- Returns the score with the frozen reward model: $r = r_\psi(x \oplus y)$.

6. Implement the per-step reward (equation 7): place r^{task} at the last response token; add $r_{i,t}^{\text{KL}}$ at every response token.

C3.3 GAE and Advantage Computation

7. Implement a function to compute GAE following equation (8). Use $\gamma = 1.0$ (no discounting within a response).
8. Standardize advantages batch-wide: $\tilde{A}_t = (A_t - \mu_A)/(\sigma_A + 10^{-8})$.
9. Compute the λ-return $V^{\text{GAE}}(s_t) = V^{\text{old}}(s_t) + A_t^{\text{GAE}}$.

C3.4 PPO Update

10. Re-compute $\log \pi_\theta(y_t \mid s_t)$ for all response tokens (forward pass).
11. Compute $\rho_t(\theta) = \exp(\log \pi_\theta - \log \pi_\theta^{\text{old}})$ using the cached π_θ^{old} log-probs.
12. Implement L_{clip} (equation 9), L_V, and the KL monitoring term.
13. Combine into L_{PPO} and take a gradient step.
14. Run **4 mini-batch epochs** over each collected rollout batch.
15. Log every step: mean reward, mean KL from π_{ref}, policy loss, critic loss, gradient norm.

Implementation Guidance

Memory management for PPO with separate policy and value models:

- Three models are used: policy (SmolLM2/SmolLM3, trainable LoRA), value model (Llama-3.2-1B-Instruct + scalar head, trainable), and frozen reference policy (π_{ref}, same as policy with adapters disabled). The frozen reward model (also Llama-3.2-1B-Instruct) is only queried during rollout.
- During rollout: run all four models under `torch.no_grad()` (except no-grad is already ensured for frozen models). Clear the KV cache with `past_key_values=None` between prompts.
- Cache rollout data (log-probs, value estimates, rewards, token ids) as `float32` tensors on **CPU** if VRAM is tight; move to GPU batch-by-batch during the update step.
- If OOM during the update: reduce prompts per step first, then reduce `max_new_tokens`, then enable 8-bit quantization on the frozen reward model.

Required sanity checks before full training:

1. **GAE unit test:** For a hand-crafted trajectory ($T = 3$, $r = [0.05, -0.02, 1.6]$, $V^{\text{old}} = [1.5, 1.55, 1.45]$, $\gamma = \lambda = 1$), compute A_t^{GAE} by hand and verify your implementation matches.

2. **Ratio test:** At the *start* of the first PPO epoch (no updates yet), all $\rho_t = 1.0$. Verify this by printing the min/max of ρ_t.

3. **Clipping test:** Artificially set $\rho_t = 1.5$ and $A_t = 1.0$ with $\varepsilon = 0.2$; verify the loss equals $(1+\varepsilon) \cdot A_t = 1.2$ (clipped) and $\nabla = 0$ with respect to π_θ.

Common Pitfalls & Debugging

- **Do not recompute π_θ^{old} log-probs** during PPO update epochs. They must be cached at rollout time. Recomputing would give $\rho_t = 1.0$ by definition at the start of every epoch, breaking the ratio mechanism entirely.
- **Log-prob of the sampled token, not the full distribution**: your ratio uses $\log \pi_\theta(y_t \mid s_t)$, the scalar log-prob of the *actually generated* token, not the log of the full softmax distribution.
- **Detach value targets**: when computing L_V, the GAE target V^{GAE} must be detached from the computation graph.

4.8 Task C4: DPO Implementation

Task C4: Direct Preference Optimization

Objective: Implement DPO as an offline, reward-model-free alignment method.

C4.1 Log-Probability Computation

1. Implement a function that returns $\log \pi_\theta(y \mid x) = \sum_{t=t_{\text{start}}}^{T} \log \pi_\theta(y_t \mid x, y_{<t})$.
   - Perform one teacher-forced forward pass over the full $(x \oplus y)$ sequence.
   - Gather log-probs only for the *response* tokens (indices t_{start} to T).
   - Sum them to get the scalar sequence log-prob.
2. Verify: for a fixed (x, y^+), the function should return the same value whether y^+ is padded on the left or right (up to the attention mask).

C4.2 DPO Loss

3. Implement the DPO policy using equation (10).
4. Compute Δ_θ and Δ_{ref} using `sequence_log_prob` called on both the trainable policy and the frozen π_{ref}.
5. **The reference model must be frozen**. Wrap its forward pass with torch.no_grad().

6. Log per step: DPO loss, implicit reward margin $z = \beta(\Delta_\theta - \Delta_{\text{ref}})$, preference accuracy (fraction where $\pi_\theta(y^+ \mid x) > \pi_\theta(y^- \mid x)$).

C4.3 Training Loop

7. Train for 1 epoch (baseline config: batch 8, grad accum 4, $\beta = 0.1$).

8. Every 25 steps, evaluate: RM score on 200 held-out prompts (greedy decoding), KL from π_{ref}, preference accuracy on held-out test pairs.

Implementation Guidance

Tokenization for DPO: Tokenize `prompt + chosen_response` and `prompt + rejected_response` as two separate sequences. Pad both to the same maximum length in the batch. Record the index `response_start` for each item (length of the tokenized prompt so you know where to begin summing log-probs).
Sanity checks:

1. At initialization (before DPO updates), $\Delta_\theta \approx \Delta_{\text{ref}}$ so $z \approx 0$ and $\sigma(z) \approx 0.5$, meaning preference accuracy should be $\approx 50\%$. Verify this.

2. After 100 steps, preference accuracy on training data should be rising above 55–60%. If stuck at 50%, check that π_{ref} is truly frozen and that response-token masking is correct.

Common Pitfalls & Debugging

- **Mask padding in log-prob sum**: include only non-padding response tokens in $\sum_t \log \pi_\theta(y_t \mid \cdot)$. Including padding tokens inflates or deflates the log-prob in a length-dependent way that destroys the loss signal.

- **Length imbalance**: if chosen responses are systematically longer than rejected ones (common in HH-RLHF), verbosity bias (Problem 3.3) will manifest. Observe this empirically: track mean length of responses the model learns to prefer.

4.9 Task C5: GRPO Implementation

Task C5: Group Relative Policy Optimization

Objective: Implement GRPO as a critic-free online alignment method.

C5.1 Group Roll-out and Advantage

1. Implement the GRPO rollout:

   - For each prompt x_b in the batch, sample K completions using the current policy with temperature sampling.
   - Score all K completions: $r_{b,k} = r_\psi(x_b \oplus y_{b,k})$.
   - Compute group mean $\mu_b = \frac{1}{K}\sum_k r_{b,k}$ and group advantage $A_{b,k} = r_{b,k} - \mu_b$.

- Broadcast $A_{b,k}$ to all response tokens in completion k.

2. Standardize advantages across all valid (non-padding) response tokens in the batch: $\tilde{A}_{b,k,t} = (A_{b,k} - \mu_A)/(\sigma_A + 10^{-8})$.

3. Log: fraction of batches where all K rewards are identical (degenerate batch, zero gradient). If $> 30\%$ of batches are degenerate, your reward model may not discriminate well enough.

C5.2 GRPO Update

4. Compute per-token log-probs of the current policy π_θ and cached π_θ^{old}.

5. Compute $\rho_{k,t}(\theta) = \exp(\log \pi_\theta(y_{k,t} \mid s_{k,t}) - \log \pi_\theta^{\text{old}}(y_{k,t} \mid s_{k,t}))$.

6. Implement L_{GRPO} from equation (12) with the $\frac{1}{T_k}$ normalization.

7. Add the per-token KL term: for each response token, compute $\text{KL}(\pi_\theta(\cdot \mid s_{k,t}) \| \pi_{\text{ref}}(\cdot \mid s_{k,t}))$ using the full vocabulary log-prob vectors:

$$\text{KL}_t = \sum_v \pi_\theta(v \mid s_t) \big[\log \pi_\theta(v \mid s_t) - \log \pi_{\text{ref}}(v \mid s_t)\big].$$

Note on VRAM: if computing full-vocabulary KL is too expensive, use the Monte Carlo approximation: $\text{KL}_t \approx \log \pi_\theta(y_t \mid s_t) - \log \pi_{\text{ref}}(y_t \mid s_t)$ (the sampled-token approximation).

Common Pitfalls & Debugging

- **Cached π_θ^{old}:** same as PPO—do not re-run the policy at rollout time inside the update step. Ratios must use the log-probs from when the completions were generated.
- **Degenerate batches:** if all K completions score identically, $A_{b,k} = 0$ and the gradient contribution from that prompt is zero. This is expected but should be rare with a well-trained RM.
- **KL approximation:** if you use the sampled-token KL approximation, note that it is an *unbiased* estimate of the true KL but has high variance for tokens with low probability under π_θ.

4.10 Task C6: RLVR — RL with Verifiable Rewards

Task C6: RLVR on GSM8K

Objective: Replace the learned reward model with a *verifiable* reward and apply GRPO to a math reasoning task.

C6.1 Dataset and Verifiable Reward

1. Load the GSM8K dataset (`openai/gsm8k`, `main` config) This dataset contains ∼7.5k grade-school math word problems, each with a step-by-step solution and a final numeric answer.

2. Implement a function that parses the final numeric answer from a model-generated solution string. The answer may appear after "`The answer is`", "`####`", or at the end of a boxed expression. Your parser should:

   - Strip commas from numbers (e.g., "1,234" → 1234).
   - Handle negative numbers and simple decimals.
   - Return `None` if no valid number is found (treat as incorrect).

3. Implement the verifiable reward:

$$r_v(x, y) = \mathbf{1}\big[\texttt{extract_answer}(y) = g(x)\big] \in \{0, 1\},$$

   where $g(x)$ is the gold answer extracted from the GSM8K ground-truth solution.

4. **Verify:** run `extract_answer` on 20 gold solutions from the dataset and confirm $r_v = 1$ for all. Run on 20 obviously wrong strings and confirm $r_v = 0$.

C6.2 Prompt Formatting

5. Format each GSM8K problem as a chat prompt for your SmolLM2/SmolLM3 model. A simple template:

```
"Solve the following math problem step by step.
At the end, write your final answer as a single number.

Problem: {question}
Solution:"
```

6. Keep prompts short: truncate the question to at most 200 tokens. Use `max_new_tokens = 256` for generation to allow enough room for step-by-step reasoning.

7. Split: use the GSM8K `train` split (∼7.5k) for RLVR training and the `test` split (∼1.3k) for evaluation.

C6.3 RLVR Training (GRPO with r_v)

8. Re-use your GRPO implementation from Task C5. The only change is the reward signal: replace the RM call with r_v. No reward model is needed.

9. Initialize the policy from the SFT checkpoint (Task C2, trained on HH-RLHF). The policy has not seen GSM8K during SFT; this tests whether RLVR can teach new capabilities from scratch using only binary feedback.

10. Configure RLVR as specified in the baseline: 300 steps, 8 prompts per step, $K = 4$ roll-outs, $\beta = 0.05$, $\varepsilon = 0.2$, max new tokens 256.

11. Log every 25 steps:
    - **pass@1**: fraction of greedy-decoded test solutions with $r_v = 1$.
    - Mean group reward μ_b across all prompts in the batch.
    - Fraction of degenerate batches (all-same reward $\Rightarrow$ zero gradient).
    - Mean response length over time (to detect verbosity drift).
    - KL from π_{ref} (your HH-RLHF SFT checkpoint).

C6.4 Analysis

12. **Format compliance:** Beyond answer correctness, track the fraction of responses that contain a final numeric answer at all (regardless of correctness). Does the model learn to produce structured answers over training, even when r_v only rewards the final value?

13. **Degenerate batches:** Because $r_v \in \{0, 1\}$, all-correct or all-incorrect batches give zero gradient. Plot the degenerate batch fraction over training. At what difficulty regime (easy/hard problems) does this occur most?

14. **Credit assignment:** r_v is sparse—it is received only at the end of a 256-token sequence. Compute the fraction of tokens that carry a nonzero gradient signal per batch. How does this compare to PPO with dense per-token KL rewards? Relate your observation to Problem 4.3(b).

15. **Sample table:** Show 5 GSM8K test problems with your model's generated solution (after RLVR) and ground truth. Include at least 2 cases where the model gets the right answer and 2 where it fails. Note any interesting failure patterns (e.g., correct intermediate steps but wrong final arithmetic).

Implementation Guidance

Answer extraction is harder than it looks. GSM8K solutions end with "`#### {answer}`", but model-generated solutions may use different formats. A robust extractor may be needed. Invalid extractions (returning `None`) should always count as incorrect ($r_v = 0$).
VRAM note: RLVR uses `max_new_tokens = 256`, which is 2× the limit used in PPO/GRPO for HH-RLHF. This increases KV-cache memory. If you encounter OOM, reduce to `max_new_tokens = 128` and note this.

Common Pitfalls & Debugging

- **Degenerate batches are common early in RLVR.** If the model cannot solve any GSM8K problem initially, all K completions get $r_v = 0$ and all advantages are zero. The model learns nothing until it produces at least one correct completion per group. If pass@1 is still 0% after 100 steps, verify your answer extractor is not too strict and that the prompt format matches what the model was SFT-trained on.

- **Do not initialize from the PPO/GRPO/DPO HH-RLHF checkpoint.** Always start RLVR from the plain SFT checkpoint. This ensures a fair comparison and avoids

artifacts from HH-RLHF fine-tuning.

- π_{ref} **for RLVR** is the same HH-RLHF SFT checkpoint you used for the other methods. The KL term keeps the model from drifting too far from its language capabilities, even though the task is now math.

4.11 Task C7: Focused Ablations

Choose **at least one** of the following ablations. State a hypothesis, run the experiment, and explain whether results support it. Keep all unreported hyperparameters fixed.

Task C7: Ablations

1. **KL coefficient sweep (PPO or GRPO):** Train with $\beta \in \{0, 0.05, 0.1, 0.5\}$ for 200 steps each. Report RM score (proxy for reward hacking) and KL from π_{ref} at end of training. Identify the value where reward gaming first appears (high RM score but low response quality on manual inspection).

2. **Clipping ablation (PPO or GRPO):** Train with $\varepsilon \in \{0.05, 0.2, 0.5, \infty\}$ (unclipped CPI for ∞). Report: gradient norm variance per step, final RM score on held-out prompts. Plot training instability (variance in per-step reward) vs. ε.

3. **Roll-out group size K (GRPO):** Train with $K \in \{1, 2, 4, 8\}$, keeping total reward model calls per step constant at $8K_{\text{base}}$ (i.e. adjust prompts per step accordingly). Report: fraction of degenerate batches vs. K, final RM score. Explain theoretically why larger K reduces variance in μ_b.

4. **DPO β sweep:** Train with $\beta \in \{0.01, 0.1, 0.5, 1.0\}$. Report: preference accuracy on test pairs, KL from π_{ref}, RM score on greedy outputs. Is there a β that maximizes test-set RM score? Relate your finding to the regularization effect identified in Problem 3.2(a).

4.12 Task C8: Evaluation

Task C8: Evaluation

Objective: Quantify and compare SFT, SFT+PPO, SFT+DPO, SFT+GRPO, and SFT+RLVR.

Required Metrics

1. **RM win-rate vs. SFT:** For 200 held-out prompts, generate greedy responses from the SFT model and from each aligned model. Feed both to the frozen reward model. Report: fraction of prompts where the aligned model's response scores higher than the SFT baseline (win-rate). This is the primary quality metric.

2. **KL from π_{ref}:** $\mathbb{E}_x[\mathrm{KL}(\pi_\theta(\cdot \mid x) \| \pi_{\text{ref}}(\cdot \mid x))]$ at end of training. Compute using the Monte Carlo approximation over a fixed set of 200 prompts.

3. **Sample response table:** For 5 held-out prompts, view responses from SFT, PPO,

DPO, GRPO side by side. Include at least 2 prompts where methods disagree (one generates a better response than others). Include the RM score for each response.

4. **Resource table:** Log for each method: peak VRAM (GB), time per update step (seconds), and total training time. Comment on the compute trade-off vs. quality.

Analysis Questions

5. **Reward vs. quality gap:** Is RM win-rate a reliable proxy for response quality as judged by you? Pick 5 responses where RM score is high but quality seems low, or vice versa. What does this reveal about the reward model?

6. **Online vs. offline comparison:** DPO uses a fixed offline preference dataset while PPO/GRPO query the RM online. After the same number of RM score computations (as a proxy for annotation cost), which method achieves higher win-rate? Comment on what this says about sample efficiency.

7. **KL as alignment measure:** A small KL from π_{ref} means the model stays close to the SFT policy. A large KL might indicate reward hacking or improved specialization. Based on your results, does lower KL correlate with better human-perceived quality? Use your sample table as evidence.

Implementation Guidance

Common evaluation pitfalls:

- Reporting only RM score (gameable) without manual quality inspection.
- Comparing methods at different numbers of gradient steps. For fairness, compare at equal total RM calls (reward model query budget).
- Not reporting KL: two methods with similar RM scores but very different KL values have fundamentally different alignment properties.
- Forgetting to use greedy decoding (temperature = 0) for evaluation, which would introduce sampling noise across runs.

5 Appendix

5.1 Mathematical Reference

RLHF Objective and Optimal Policy

$$J(\theta) = \mathbb{E}_{x,y}\big[r_\psi(x,y)\big] - \beta\,\mathbb{E}_x\big[\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}})\big], \qquad \pi^*(y \mid x) = \frac{\pi_{\mathrm{ref}}(y \mid x)\, e^{r_\psi(x,y)/\beta}}{Z(x)}.$$

PPO Per-Step Reward and GAE

$$r_{i,t} = r_i^{\mathrm{task}} \cdot \mathbf{1}[t = T_i] - \beta\big[\log \pi_\theta^{\mathrm{old}}(a_t \mid s_t) - \log \pi_{\mathrm{ref}}(a_t \mid s_t)\big],$$

$$\delta_t = r_t + \gamma V_\phi^{\mathrm{old}}(s_{t+1}) - V_\phi^{\mathrm{old}}(s_t), \qquad A_t^{\mathrm{GAE}} = \textstyle\sum_{k\geq 0}(\gamma\lambda)^k \delta_{t+k}, \qquad V^{\mathrm{GAE}}(s_t) = V_\phi^{\mathrm{old}}(s_t) + A_t^{\mathrm{GAE}}.$$

PPO Clipped Surrogate (Full Loss)

$$L_{\mathrm{clip}} = \mathbb{E}_t\big[\min(\rho_t \tilde{A}_t,\ \mathrm{clip}(\rho_t, 1{-}\varepsilon, 1{+}\varepsilon)\tilde{A}_t)\big], \qquad L_{\mathrm{PPO}} = -L_{\mathrm{clip}} + c_V L_V - c_{\mathrm{ent}} H(\pi_\theta) + c_{\mathrm{KL}} L_{\mathrm{KL}}.$$

DPO Loss

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y^+,y^-)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\mathrm{ref}}(y^+ \mid x)} - \beta \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\mathrm{ref}}(y^- \mid x)}\right)\right].$$

GRPO Loss and Variants

$$A_{b,k} = r_{b,k} - \tfrac{1}{K}\textstyle\sum_j r_{b,j}, \quad L_{\mathrm{GRPO}} = -\tfrac{1}{K}\textstyle\sum_k \tfrac{1}{T_k}\sum_t \min\big(\rho_{k,t}A_{k,t},\ \mathrm{clip}(\rho_{k,t}, 1{-}\varepsilon, 1{+}\varepsilon)A_{k,t}\big) + \beta \mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}}).$$

5.2 VRAM Budget (Approximate, bfloat16)

Component	**DPO**	**GRPO**	**PPO**	**Notes**
Policy π_θ (1.1B, bf16)	2.2 GB	2.2 GB	2.2 GB	trainable (LoRA)
Reference π_{ref} (frozen)	2.2 GB	2.2 GB	2.2 GB	frozen
Reward model (frozen)	—	2.2 GB	2.2 GB	frozen
Value head	—	—	$<$ 0.1 GB	small linear
LoRA optimizer states	$\sim$0.1 GB	$\sim$0.1 GB	$\sim$0.1 GB	rank 8
Activations + KV cache	$\sim$1 GB	$\sim$2 GB	$\sim$2 GB	seq len 512
Total (estimate)	$\sim$6 GB	$\sim$9 GB	$\sim$11 GB	T4 (15 GB) feasible

Tips to reduce VRAM further:

- Use `bitsandbytes` 8-bit quantization on frozen models (π_{ref}, RM): `load_in_8bit=True` halves their footprint.
- Reduce `max_new_tokens` to 64 during ablations.
- Enable `torch.backends.cuda.matmul.allow_tf32 = True`.
- Offload π_{ref} to CPU and move to GPU only for the KL computation step.

5.3 Debugging Checklist

Symptom	**Likely cause and fix**
PPO ratio $\rho_t \neq 1.0$ at start of epoch 1	You are recomputing π_θ^{old} log-probs; must cache them at rollout time.
DPO preference accuracy stuck at 50%	Reference model is not frozen, or response tokens are not correctly masked in log-prob sum.
RM preference accuracy below 55%	Check scalar head reads last non-pad token; check prompt is concatenated with response.
GRPO: all advantages zero	All K completions got the same reward (degenerate batch); lower temperature or increase K.
OOM during PPO update	Cache rollout data on CPU; reduce prompts per step; enable gradient checkpointing.
SFT perplexity suspiciously low	Loss is computed over prompt tokens; check label masking (`labels[prompt]=-100`).
Generation produces only EOS or padding	Padding side is right instead of left for a decoder-only model; fix `padding_side=``left''`.